

RAG (Retrieval-Augmented Generation)

Grounding LLM answers in real, current, external knowledge — embeddings, chunking, similarity search, vector databases, retrieval, re-ranking, and hybrid search.

DIFFICULTY LEVEL

Intermediate

ESTIMATED READING TIME

55-70 minutes

ESTIMATED PRACTICE TIME

100-130 minutes

PREREQUISITES

Modules 01-06

LEARNING OUTCOMES

By the end of this module, you will be able to: explain how embeddings turn text into searchable vectors; chunk documents effectively for retrieval; perform similarity search over a vector database; build a basic retrieval pipeline; apply re-ranking to improve result quality; and combine keyword and semantic search via hybrid search.

Table of Contents

1.	Introduction	
2.	Before You Begin	
3.	Main Lesson	
3.1	Embeddings	
3.2	Chunking	
3.3	Similarity Search	
3.4	Vector Databases	
3.5	Retrieval	
3.6	Re-ranking	
3.7	Hybrid Search	
4.	Visual Learning	
5.	Real World Applications	
6.	Practical Examples	
7.	Code Examples	
8.	Common Mistakes	
9.	Best Practices	
10.	Hands-on Activities	
11.	Mini Project	
12.	Review Questions	
13.	Cheat Sheet	
14.	Key Takeaways	
15.	Additional Resources	

1. Introduction

What is this topic?

RAG (Retrieval-Augmented Generation) is the technique of fetching relevant external information at request time and feeding it into the model's context before it generates an answer — the concrete, most common implementation of "retrieval context" from Module 05.

Why is it important?

An LLM's trained-in knowledge is frozen at training time and can't include your private documents, your company's data, or anything that changed since training. RAG solves this without retraining the model — by retrieving the right information and simply putting it in the prompt.



Why should AI Engineers learn it?

RAG is one of the most widely deployed AI engineering patterns in production — it's usually the difference between "a chatbot that hallucinates confidently" and "a chatbot that reliably answers from your actual documents." It's foundational infrastructure for most knowledge-based AI products.

Where is it used in the real world?

Documentation chatbots, internal company knowledge assistants, legal/medical document Q&A tools, and customer support bots grounded in a specific product's manuals all rely on RAG.

Why is it relevant today?

Even as context windows grow (Module 01, 3.6), RAG remains essential for knowledge bases too large to fit in any context window, and for keeping answers grounded in the freshest, most relevant information rather than paying to reprocess entire archives on every request.

Note

This module is the deep technical dive behind Module 05's "Retrieval Context" concept — if that section felt abstract, this module makes it concrete and buildable.

2. Before You Begin

RECAP

Retrieval context (Module 05, 3.7) — RAG is the concrete system that implements this concept using embeddings and vector search.

RECAP

Hallucinations (Module 01, 3.11) — RAG is one of the strongest practical mitigations, since the model answers from retrieved facts instead of memorized (and possibly wrong) training data.



Tip

RAG quality is a pipeline problem — bad chunking or bad retrieval will produce bad answers even with the best possible prompt and the best possible model. Debug the pipeline stage by stage.

3. Main Lesson

Each concept follows: **Definition** → **Simple Explanation** → **Analogy** → **Everyday Example** → **Why It Matters** → **Common Mistakes** → **Best Practices** → **Summary**.

3.1 Embeddings

DEFINITION

An embedding is a numeric vector representation of text (or images) that captures its meaning, positioned in a high-dimensional space where similar meanings end up close together.

SIMPLE EXPLANATION

A separate embedding model converts a chunk of text into a list of numbers (e.g., 1,536 numbers). Texts with similar meaning produce vectors that are mathematically close to each other, even if they don't share the exact same words.

```
"How do I reset my password?" → [0.12, -0.87, 0.34, ...]
"Steps to change my login PIN" → [0.15, -0.81, 0.29, ...] (close! similar meaning)
"What's the weather today?" → [0.91, 0.02, -0.55, ...] (far — unrelated meaning)
```

💡 Like GPS coordinates for meaning — two places with similar coordinates are physically close, and two texts with similar embeddings are "close" in meaning, even using totally different words.

EVERYDAY EXAMPLE

"Reset my password" and "change my login PIN" produce embeddings close to each other despite sharing almost no exact words — because their meanings are similar.

WHY IT MATTERS

Embeddings are what make semantic search possible — finding relevant content by *meaning*, not just exact keyword matches, which is the core capability RAG depends on.

COMMON MISTAKES

Mixing embeddings from different embedding models in the same search index — vectors from different models aren't comparable to each other.

BEST PRACTICES

Use the same embedding model consistently for both your stored documents and incoming queries; re-embed everything if you switch embedding models.

SUMMARY

Embeddings turn text into meaning-aware numeric vectors — the foundational data type that all of RAG's retrieval steps operate on.

3.2 Chunking

DEFINITION

Chunking is splitting large documents into smaller, manageable pieces before embedding and storing them, since embedding an entire document as one vector loses too much specific detail.

SIMPLE EXPLANATION

A 50-page manual embedded as a single vector can only capture a vague, blended sense of the whole document. Chunking into paragraph- or section-sized pieces lets each piece get its own precise embedding, so retrieval can pinpoint the exact relevant section.

💡 Like indexing a book by individual pages or sections instead of just one entry for "the whole book" — a page-level index lets you jump straight to the relevant page instead of being told only "yes, it's somewhere in this book."

EVERYDAY EXAMPLE

Splitting a company handbook into chunks by section (e.g., "Leave Policy," "Expense Reimbursement") so a question about vacation days retrieves specifically the leave policy section, not the whole handbook.

WHY IT MATTERS

Chunk size and boundaries directly determine retrieval precision — too large and you retrieve noisy, unfocused chunks; too small and you lose necessary surrounding context.

COMMON MISTAKES

Chunking purely by a fixed character count with no regard for natural boundaries (sentences, paragraphs, sections), which can cut a sentence or idea awkwardly in half.

BEST PRACTICES

Chunk along natural boundaries (paragraphs, headers) where possible; use overlapping chunks (e.g., 10-20% overlap) so ideas spanning a chunk boundary aren't lost entirely.

SUMMARY

Chunking breaks documents into retrieval-sized pieces — the granularity choice that most directly shapes retrieval precision.

3.3 Similarity Search

DEFINITION

Similarity search is the process of finding which stored embeddings are mathematically closest to a query's embedding, typically using a metric like cosine similarity.

SIMPLE EXPLANATION

You embed the user's query the same way you embedded your documents, then search your vector store for the stored chunks whose vectors are nearest to the query vector — those are your most semantically relevant results.

```
Query: "How do I reset my password?"
  | embed
  ▼
[0.12, -0.87, 0.34, ...]
  | compare to all stored chunk vectors
  ▼
Closest matches: Chunk #14 (0.97 similarity)
                  Chunk #3  (0.85 similarity)
                  Chunk #29 (0.81 similarity)
```

💡 Like finding the closest restaurants to your current GPS location on a map — you're not looking for an exact address match, just the nearest points in space.

EVERYDAY EXAMPLE

A support search that returns the "Reset Password" help article for the query "I can't log in, forgot my PIN" — despite no exact keyword overlap, because the embeddings are close.

WHY IT MATTERS

Similarity search is what actually finds candidate content at query time — the engine underneath every RAG retrieval step.

COMMON MISTAKES

Not setting a relevance threshold, so even poor matches (low similarity) get returned and injected into context, diluting answer quality.

BEST PRACTICES

Set a minimum similarity threshold; return only the top-K most similar results (commonly 3-5) rather than everything above a loose bar.

SUMMARY

Similarity search finds the closest-meaning stored chunks to a query — the core retrieval mechanism behind semantic search.

3.4 Vector Databases

DEFINITION

A vector database is a specialized database designed to store embeddings and perform fast similarity search over potentially millions of vectors.

SIMPLE EXPLANATION

Regular databases are optimized for exact matches and simple filters. Vector databases are optimized for "find the nearest neighbors in high-dimensional space" — a fundamentally different kind of query that needs specialized indexing (like HNSW or IVF) to stay fast at scale.

💡 Like the difference between a phone book (sorted alphabetically, great for exact name lookup) and a "find the 5 nearest coffee shops" map feature — different data structures optimized for very different kinds of queries.

EVERYDAY EXAMPLE

Popular vector databases include Pinecone, Weaviate, Chroma, and pgvector (a Postgres extension) — each storing embeddings alongside the original text and metadata for fast similarity search.

WHY IT MATTERS

Without a vector database (or vector index), similarity search against a large document collection would require comparing the query to every single stored vector one by one — far too slow at real-world scale.

COMMON MISTAKES

Choosing a vector database based purely on hype rather than actual scale needs — a small knowledge base often doesn't need a dedicated, heavyweight vector database at all.

BEST PRACTICES

For small-scale projects, a simple in-memory or lightweight solution (like Chroma or even a flat file) is often enough; reserve large managed vector databases for genuinely large-scale production needs.

SUMMARY

Vector databases make similarity search fast at scale — essential infrastructure once your document collection grows beyond what naive comparison can handle quickly.

3.5 Retrieval

DEFINITION

Retrieval is the end-to-end step of taking a user query, searching the vector database, and returning the set of chunks that will be injected into the prompt as context.

SIMPLE EXPLANATION

Retrieval combines embedding the query, running similarity search, and applying any filters (metadata, date ranges, permissions) to produce the final set of chunks that actually gets used — it's the orchestration layer tying embeddings, chunking, and the vector database together.

Embed Query



Search Vector DB



Apply Filters



Top-K Chunks

💡 Like a librarian who doesn't just grab the nearest-looking books, but also checks which section you're allowed into, which edition is current, and narrows to the specific handful most useful to your exact question.

EVERYDAY EXAMPLE

A support bot's retrieval step might filter to only the current product version's docs and only articles the user's account tier has access to, before even running similarity search.

WHY IT MATTERS

Retrieval quality is the single biggest lever on RAG answer quality — even a perfect prompt can't compensate for the wrong chunks being retrieved in the first place.

COMMON MISTAKES

Treating retrieval as "just similarity search" and ignoring important filters (recency, permissions, source reliability) that should narrow candidates first.

BEST PRACTICES

Combine similarity search with metadata filters where relevant; monitor retrieval quality separately from generation quality when debugging bad answers.

SUMMARY

Retrieval is the full pipeline stage that turns a query into the final context chunks — the layer where embeddings, chunking, and the vector database all come together.

🔧 3.6 Re-ranking

DEFINITION

Re-ranking is a second-pass step that takes an initial set of retrieved candidates and reorders (or filters) them using a more precise, often more expensive, relevance model.

SIMPLE EXPLANATION

Vector similarity search is fast but somewhat approximate. Re-ranking takes the top 20-50 candidates from that first pass and scores each one more carefully against the query — often using a specialized "cross-encoder" model — to surface the truly best few before they reach the prompt.



💡 Like a job application process: a fast resume keyword scan narrows 500 applicants to 30 (similarity search), then a more careful, time-intensive interview process picks the best 5 from those 30 (re-ranking).

EVERYDAY EXAMPLE

Retrieving 30 candidate chunks via fast vector similarity, then using a cross-encoder re-ranker to pick the truly best 5 before injecting them into the prompt.

WHY IT MATTERS

Re-ranking meaningfully improves precision over similarity search alone, since it can consider the query and each candidate together in more detail than a pure vector-distance comparison allows.

COMMON MISTAKES

Re-ranking the entire document collection instead of just the initial candidate set — re-ranking is only efficient as a second pass over a small shortlist.

BEST PRACTICES

Use similarity search to cheaply narrow to ~20-50 candidates, then re-rank only that shortlist down to the final top few for the prompt.

SUMMARY

Re-ranking adds a more precise second pass on top of similarity search's fast first pass —

a strong quality upgrade for retrieval-sensitive applications.

❌3.7 Hybrid Search

DEFINITION

Hybrid search combines semantic (embedding-based) search with traditional keyword search, merging both result sets to get the benefits of each.

SIMPLE EXPLANATION

Semantic search is great at understanding meaning but can miss exact terms that matter (product codes, specific names, acronyms). Keyword search is great at exact matches but misses paraphrased meaning. Hybrid search runs both and combines the results, usually with a weighted scoring formula.

💡 *Like searching for a job candidate using both "similar skill profile" matching AND an exact required-certification filter — one alone might miss great candidates that the other alone would catch.*

EVERYDAY EXAMPLE

Searching "error code E4021 troubleshooting" benefits from exact keyword matching on "E4021" (which embeddings might blur together with similar-looking codes) combined with semantic understanding of "troubleshooting."

WHY IT MATTERS

Pure semantic search alone can underperform on queries with exact identifiers, codes, names, or rare technical terms — hybrid search closes that specific gap.

COMMON MISTAKES

Assuming semantic search alone is always sufficient — for domains full of exact codes, IDs, or specific terminology, pure semantic search often underperforms noticeably.

BEST PRACTICES

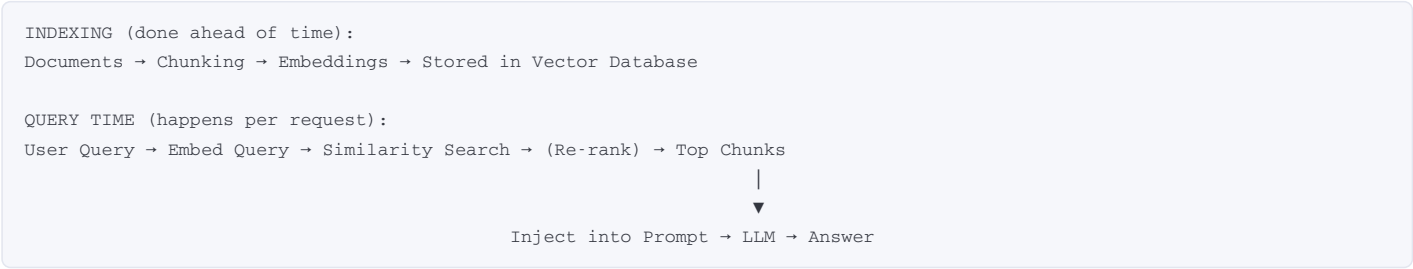
Use hybrid search especially for domains with structured identifiers (legal citations, product SKUs, error codes) alongside natural language content.

SUMMARY

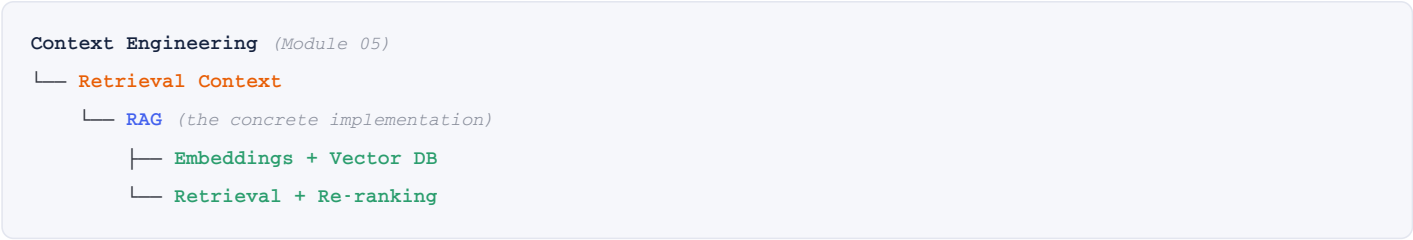
Hybrid search combines semantic and keyword search to cover both paraphrased meaning and exact-term matches — often outperforming either approach alone.

4. Visual Learning

The Full RAG Pipeline



Where RAG Fits



Search Type Comparison

Search Type	Strength	Weakness
Keyword search	Exact terms, codes, names	Misses paraphrased meaning
Semantic (embedding) search	Understands meaning/paraphrase	Can miss exact identifiers
Hybrid search	Covers both strengths	More engineering complexity

5. Real World Applications

Product	How these fundamentals show up
Documentation chatbots (e.g., docs.claude.com-style assistants)	Chunk docs by section, embed, retrieve top matches for each user question.
Internal company knowledge assistants	Hybrid search combines exact policy-code lookup with semantic understanding of employee questions.
Legal/medical document Q&A tools	Re-ranking is critical here — precision matters enormously, so a fast-then-precise two-stage pipeline is standard.
E-commerce product search	Hybrid search catches both exact SKU/model number queries and natural-language "comfortable running shoes" style queries.

6. Practical Examples

Beginner: Feeling embedding similarity

Write 5 sentence pairs — some clearly similar in meaning, some clearly unrelated. Predict which pairs would have high vs. low embedding similarity before checking with an actual embedding API or tool.

Beginner: Manual chunking

Take a 2-page document and manually split it into chunks by natural section boundaries. Compare against splitting it by a fixed character count — notice where the fixed-count approach cuts awkwardly.

Intermediate: Retrieval quality check

Build a tiny 10-document knowledge base (even just 10 short paragraphs), embed and store them, then test 5 queries and manually judge whether the top-3 retrieved results are actually relevant.

Advanced: Hybrid search comparison

Design a query containing a specific code/identifier (e.g., "error E4021"). Compare results from pure semantic search vs. a simple keyword filter added on top — observe the difference in whether the exact code is found.

7. Code Examples

🐍Python — Simple RAG pipeline (chunk, embed, retrieve, answer)

```
# pip install anthropic numpy
import anthropic
import numpy as np

client = anthropic.Anthropic(api_key="YOUR_API_KEY")

# --- Step 1: Chunking (simplified, by paragraph) ---
document = """Password resets: go to Settings > Security > Reset Password.
Billing questions: contact billing@example.com for invoice issues.
Shipping times: standard shipping takes 5-7 business days."""
chunks = [c.strip() for c in document.split("\n") if c.strip()]

# --- Step 2: Embeddings (pseudo-code; use a real embedding API/model) ---
def embed(text):
    # In real code: call an embedding model (e.g., voyage-3, text-embedding-3)
    return fake_embedding_model(text) # placeholder

chunk_embeddings = [embed(c) for c in chunks]

# --- Step 3: Similarity search (cosine similarity) ---
def cosine_similarity(a, b):
    return np.dot(a, b) / (np.linalg.norm(a) * np.linalg.norm(b))

def retrieve(query, top_k=2):
    q_emb = embed(query)
    scores = [cosine_similarity(q_emb, c) for c in chunk_embeddings]
    top_indices = np.argsort(scores)[::-1][:top_k]
    return [chunks[i] for i in top_indices]

# --- Step 4: Inject retrieved chunks into the prompt ---
query = "How long does shipping take?"
context = "\n".join(retrieve(query))

prompt = f"""<context>
{context}
</context>

Using only the context above, answer: {query}"""

response = client.messages.create(
    model="claude-sonnet-4-6", max_tokens=200,
    messages=[{"role": "user", "content": prompt}]
)
print(response.content[0].text)
```

📄JavaScript — Retrieval + re-ranking sketch

```
// npm install @anthropic-ai/sdk
import Anthropic from "@anthropic-ai/sdk";
const client = new Anthropic({ apiKey: process.env.ANTHROPIC_API_KEY });

// Step 1: fast similarity search returns a shortlist
function similaritySearch(queryEmbedding, chunkEmbeddings, topK = 20) {
    // returns indices of the topK closest chunks (implementation omitted)
}

// Step 2: re-rank the shortlist more precisely (e.g., via a cross-encoder API)
async function rerank(query, candidateChunks, topK = 5) {
    // call a reranking model/service; return the best topK, reordered
}

async function ragAnswer(query, chunks, chunkEmbeddings, queryEmbedding) {
    const shortlist = similaritySearch(queryEmbedding, chunkEmbeddings, 20)
        .map((i) => chunks[i]);
    const best = await rerank(query, shortlist, 5);

    const context = best.join("\n");
    const response = await client.messages.create({
        model: "claude-sonnet-4-6",
        max_tokens: 200,
```

```
messages: [{ role: "user", content: `Context:\n${context}\n\nQuestion: ${query}` }],
});
return response.content[0].text;
}
```

**Tip**

Always instruct the model to answer "using only the provided context" and to say when the answer isn't in the context — this keeps RAG answers grounded and avoids the model quietly falling back on possibly outdated trained-in knowledge.

8. Common Mistakes

- **Chunking by fixed character count only**, cutting ideas awkwardly across chunk boundaries.
- **Mixing embeddings from different models** in the same vector index — they aren't comparable.
- **No relevance threshold**, injecting poor matches into context and diluting answer quality.
- **Skipping re-ranking** for precision-critical applications where similarity search alone isn't precise enough.
- **Relying only on semantic search** in domains with important exact identifiers (codes, SKUs, citations).
- **Not instructing the model to stay grounded in the retrieved context**, letting it fall back on ungrounded trained knowledge.

9. Best Practices

- **Chunk along natural boundaries** with modest overlap between chunks.
- **Use the same embedding model** consistently for documents and queries.
- **Set a similarity threshold** and return a small top-K, not everything above a loose bar.
- **Add re-ranking** for applications where precision genuinely matters.
- **Use hybrid search** in domains with exact identifiers alongside natural language.
- **Explicitly instruct the model to answer from the provided context only**, and to say when it can't.

10. Hands-on Activities

Easy 5 exercises

1. Write 3 pairs of sentences and predict, before testing, which pairs would have high embedding similarity.
2. Manually chunk a short document (your choice) into 3-5 natural sections.
3. Sketch the full RAG pipeline diagram from Section 4 from memory, then compare against the original.
4. List 3 queries where keyword search would likely outperform semantic search, and 3 where the reverse is true.
5. Research one real vector database (Pinecone, Weaviate, Chroma, or pgvector) and summarize its core features in 3-4 sentences.

Intermediate 3 exercises

1. Build the simple RAG pipeline from Section 7 with a real embedding API and a small (5-10 document) knowledge base.
2. Test your pipeline with a query containing a specific code/identifier and check whether pure semantic search retrieves it correctly.
3. Add a similarity threshold to your retrieval function and observe how it changes which chunks get returned for borderline queries.

Challenge 2 exercises

1. Design (on paper) a hybrid search scoring formula that combines a keyword match score and a semantic similarity score into one ranked list.
2. Add a re-ranking step to your Section 7 pipeline (even a simplified heuristic re-ranker) and compare the final top-3 results before and after re-ranking.

11. Mini Project

Goal

Build a small "Ask My Notes" RAG tool: chunk a set of personal notes, embed and store them, retrieve relevant chunks for a question, and generate a grounded answer.

Requirements

- An Anthropic API key and an embedding API/model of your choice
- Python or JavaScript
- 5-10 short text notes of your own (study notes, journal entries, or project docs work well)

Step-by-Step Instructions

1. Chunk each note into paragraph-sized pieces.
2. Embed all chunks and store them (a simple in-memory list of vectors is fine at this scale).
3. Write a `retrieve(query, top_k)` function using cosine similarity.
4. Build a prompt that injects the top-K retrieved chunks with clear delimiters and instructs the model to answer only from that context.
5. Test with 5 questions, at least one of which has no good answer in your notes — confirm the model says so instead of guessing.

Expected Output

```
Q: "What did I decide about the database schema?"
Retrieved: [chunk 3, chunk 7]
Answer: [grounded answer citing the retrieved content]

Q: "What's the capital of France?" (not in notes)
Retrieved: [low-relevance chunks]
Answer: "I don't see this covered in the provided notes."
```

Possible Improvements

- Swap your in-memory store for a real lightweight vector database (e.g., Chroma).
- Add metadata filtering (e.g., by note date or topic tag) before similarity search.
- Add a re-ranking step and compare answer quality with and without it.

12. Review Questions

Multiple Choice (10)

1. An embedding is best described as:

- A) A compressed image B) A numeric vector representing text meaning C) A type of prompt D) A database index name

2. Chunking is done primarily to:

- A) Make documents longer B) Enable more precise, focused retrieval C) Remove all formatting D) Increase temperature

3. Similarity search typically uses which metric?

- A) Alphabetical order B) Cosine similarity (or similar distance metric) C) File size D) Word count

4. Vector databases are optimized for:

- A) Exact string matches only B) Fast nearest-neighbor search over embeddings C) Storing images only D) Running LLMs directly

5. Retrieval, as a pipeline stage, includes:

- A) Only similarity search B) Embedding the query, searching, and applying filters C) Training the model D) Tokenization only

6. Re-ranking is typically applied to:

- A) The entire document collection B) A smaller shortlist from an initial fast search C) Nothing, it's not used in RAG D) Only keyword search results

7. Hybrid search combines:

- A) Two different LLMs B) Semantic (embedding) search and keyword search C) Two vector databases D) Chunking and tokenization

8. Why might pure semantic search miss an exact error code query?

- A) Embeddings ignore all text B) Semantic similarity can blur specific exact identifiers C) Codes are always excluded from documents D) It's a random model bug

9. Using embeddings from two different models in one index causes:

- A) No issue at all B) Vectors that aren't meaningfully comparable C) Faster search D) Automatic re-ranking

10. RAG most directly helps mitigate:

- A) Token pricing B) Hallucinations, by grounding answers in retrieved facts C) Temperature tuning D) Max tokens limits

Identification (5)

1. _____ is a numeric vector representation of text capturing its meaning.

2. _____ is splitting large documents into smaller pieces before embedding.

3. _____ is a database specialized for fast nearest-neighbor search over embeddings.

4. _____ is a second-pass step that reorders retrieved candidates using a more precise relevance model.

5. _____ combines semantic and keyword search to cover both meaning and exact terms.

True or False (5)

1. Embeddings from different embedding models are always directly comparable.

2. Chunking along natural boundaries (paragraphs, sections) is generally better than a pure fixed character count.

3. Re-ranking is typically applied to the entire document collection rather than a shortlist.

4. Hybrid search can outperform pure semantic search for queries containing exact codes or identifiers.

5. RAG helps ground LLM answers in retrieved facts, reducing reliance on possibly outdated trained knowledge.

Situational (5)

1. Your RAG chatbot retrieves technically related but ultimately unhelpful chunks for many queries. What retrieval-stage fix would you try first?
2. Users searching for specific product SKUs get poor results from your semantic-only search. What should you add?
3. Your chunks are cutting important sentences in half at chunk boundaries. What chunking strategy change would help?
4. You need very high precision for a legal document Q&A tool and can afford extra latency. What extra pipeline stage would you add?
5. Your RAG bot sometimes answers confidently even when nothing relevant was actually retrieved. What prompt-level instruction is likely missing?

Answer Key

Multiple Choice: 1-B, 2-B, 3-B, 4-B, 5-B, 6-B, 7-B, 8-B, 9-B, 10-B

Identification: 1-Embedding, 2-Chunking, 3-Vector database, 4-Re-ranking, 5-Hybrid search

True or False: 1-False, 2-True, 3-False, 4-True, 5-True

Situational (sample reasoning):

1. Add a similarity threshold and/or re-ranking to filter out low-relevance chunks.
2. Hybrid search — combine keyword matching for SKUs with semantic search.
3. Chunk along natural boundaries (paragraphs/sentences) with some overlap, instead of a pure fixed character count.
4. Re-ranking — add a precise second pass over the initial retrieved shortlist.
5. An explicit instruction to answer only from the provided context and say when the answer isn't present.

13. Cheat Sheet

The Pipeline

Chunk → Embed → Store (vector DB) → Query → Similarity Search → (Re-rank) → Inject → Answer.

Re-ranking

Fast search → shortlist (~20-50) → precise re-rank → final top few.

Embeddings

Numeric vectors capturing meaning. Same model for docs and queries — never mix models.

Hybrid Search

Semantic (meaning) + keyword (exact terms) — best for domains with codes/IDs.

14. Key Takeaways

- RAG grounds LLM answers in external, retrieved facts instead of relying purely on frozen trained-in knowledge.
- Embeddings turn text into meaning-aware vectors; similarity search finds the closest matches to a query.
- Chunking strategy directly shapes retrieval precision — natural boundaries with overlap generally beat fixed character counts.
- Vector databases make similarity search fast at scale.
- Re-ranking adds a precise second pass over an initial fast shortlist, improving final result quality.
- Hybrid search combines semantic and keyword search, covering both paraphrased meaning and exact identifiers.
- RAG quality is a pipeline problem — retrieval quality usually matters more than prompt wording for the final answer's accuracy.

15. Additional Resources

Official Documentation

- Anthropic — Embeddings guidance (docs.claude.com, includes third-party embedding provider recommendations)
- Vector database docs: Pinecone, Weaviate, Chroma, and pgvector official documentation

Research Papers

- "Retrieval-Augmented Generation for Knowledge-Intensive NLP Tasks" (Lewis et al., 2020) — the original RAG paper.

Related Modules

- Module 05 — Context Engineering (the broader discipline RAG is a concrete implementation of)
- Module 08 — AI Agents (agents often use RAG as one of several tools)

Communities & Further Learning

- Vector database provider blogs and benchmarks for choosing the right tool at your specific scale.